

The messaging system in Erlang has been implemented in such a way that even if a process fails or stops executing, it can communicate the reason before it is killed, so that the remaining processes can bypass or take appropriate action. These error handling mechanisms allow the developers to create the program in such a way that processes are allowed to fail and this makes the code cleaner and shorter because most of these things are handled by the in built libraries themselves, rather than the programmer having to code them explicitly.

♦ Hot swapping or code loading

Keeping the concept of distributed computing in mind, Erlang ensures that any part of its code or program can be replaced or updated without any problems or hiccups. This makes scalability an important feature of the language and you can take advantage of multicore processors, add new nodes or change things transparently without any impact on the rest of the system. This is made possible because of the fact that a process can contain a reference to an old as well as a newer version of the same module and can cleanly make the switch at a specific time with the help of an external call that is made to the module.

The Next Step

Given that there isn't as big of a community around Erlang, as is for other languages, there are limited outlets where you are able to look at Erlang code and learn it from scratch. But some of these resources including the official documentation can help you get an overview. You can give it a test run at www.tryerlang.org in the form of an online interpreter where you can try and run some of the basic commands.

However, one of the best and easily the most popular Erlang book in our opinion is available online at <http://learnyousomeerlang.com/> written by Fred Hebert. It includes an excellent introduction and takes you step by step with the help of some funny cartoons. Some other good books include Programming Erlang by Joe Armstrong and O'Reilly's Erlang Programming by Francesco Cesarini and Simon Thompson. To get started you can either download it from <http://www.erlang.org/> or get it from your distribution's repository if you are on a linux system.

A good way to start learning would be to start looking at functional programming concepts, move towards various data structures and then look at distributed and concurrency features at the last. A sample